# ЛАБОРАТОРНА РОБОТА № 6

## «Інтерактивна сторінка: додавання елементів у DOM, робота з подіями»

**Навчальна дисципліна:** WEB-програмування
**Виконав:** студент групи КН-310 Мельник Назар
**Місто і рік:** Львів, 2026

## 1. Мета роботи

Закріпити знання про об'єктну модель документа DOM та об'єкти браузера BOM; набути практичних навичок пошуку, клонування, зміни й видалення DOM-вузлів; використати `template`, `FormData`, обробники подій і делегування подій.

## 2. Завдання

Проаналізувати надані HTML, CSS, JavaScript і SVG-спрайт. У файлі `js/index.js` реалізувати оновлення назви та опису саме поточної картки через `textContent` у блоці `if (isEditing && currentEditedElement)`. Створити тестове завдання та продемонструвати створення, редагування і видалення.

## 3. Структура сторінки та використані засоби

- `index.html` містить форму, контейнер списку, лічильник, статус і `template#task-template` із BEM-класами.
- `css/style.css` містить скидання стилів, CSS-змінні, BEM-компоненти та медіазапит для вузького екрана.
- `js/index.js` ініціалізується на `DOMContentLoaded`, використовує `getElementById`, `querySelector`, `textContent`, `cloneNode(true)`, `append`, `remove`, `FormData` та `preventDefault`.
- Один обробник `click` контейнера `#task-list` обслуговує кнопки динамічних карток за допомогою `event.target.closest(...)` — це делегування подій.
- `img/sprite.svg` містить символи кнопок, які підключено через `use`.

## 4. Відповідність вимогам методички

| Вимога | Файл / реалізація | Фактичний результат |
|---|---|---|
| Чотири файли у визначеній структурі | `index.html`, `css/style.css`, `js/index.js`, `img/sprite.svg` | Виконано |
| Inter, BEM, `template` | `index.html` | Підключено; шаблон клонується |
| CSS-змінні та стилі компонентів | `css/style.css` | CSS отримано з HTTP 200 |
| `DOMContentLoaded` і події | `js/index.js` | Ініціалізація, `submit`, `click` працюють |
| `FormData` і `preventDefault` | обробник `submit` | Значення прочитано без перезавантаження |
| Створення картки з шаблону | `cloneNode(true)`, `append` | Картка з'являється в DOM |
| Оновлення конкретної картки | `currentEditedElement.querySelector(...).textContent` | Назва й опис змінюються, нова картка не створюється |
| Делегування редагування і видалення | обробник `click` на `#task-list` | Обидві дії працюють для створеної картки |
| Підтвердження видалення | `window.confirm` | Після підтвердження картку вилучено з DOM |
| Захист від порожніх даних | `required`, `trim()`, `window.alert` | Пробільні значення відхилено |

| Вимога | Файл / реалізація | Фактичний результат |
|---|---|---|
| Посилання в `labs/index.html` | потребує спільного репозиторію курсу | Не виконано: репозиторій не надано |
| `merge`, `push`, GitHub Pages | зовнішній репозиторій і публікація | Не виконано відповідно до обмежень завдання |

# 5. Основна логіка редагування

```
if (isEditing && currentEditedElement) {
  currentEditedElement.querySelector('.task-card__title').textContent = taskData.title;
  currentEditedElement.querySelector('.task-card__desc').textContent = taskData.description;
  status.textContent = 'Завдання оновлено.';
} else {
  const fragment = taskTemplate.content.cloneNode(true);
  fragment.querySelector('.task-card__title').textContent = taskData.title;
  fragment.querySelector('.task-card__desc').textContent = taskData.description;
  taskList.append(fragment);
}
```

`currentEditedElement` зберігає посилання на вибрану картку. Тому `querySelector` виконується в межах цієї картки, а `textContent` безпечно вставляє текст без інтерпретації його як HTML.

# 6. Фактична перевірка

Синтаксис програми та тесту перевірено командою `node --check`. Локальну сторінку відкрито Chromium через Playwright. Сценарій має 25 assertions: перевірено HTTP 200 для HTML, CSS, JavaScript і SVG; початковий стан; створення; відображення даних; редагування; незмінність кількості карток під час редагування; відхилення пробільного введення; підтверджене видалення; порожній стан після видалення; browser console та page errors.

**Результат:** 25 перевірок пройдено, 0 провалено. `page errors` — 0, помилок browser console — 0, невдалих мережевих запитів — 0.

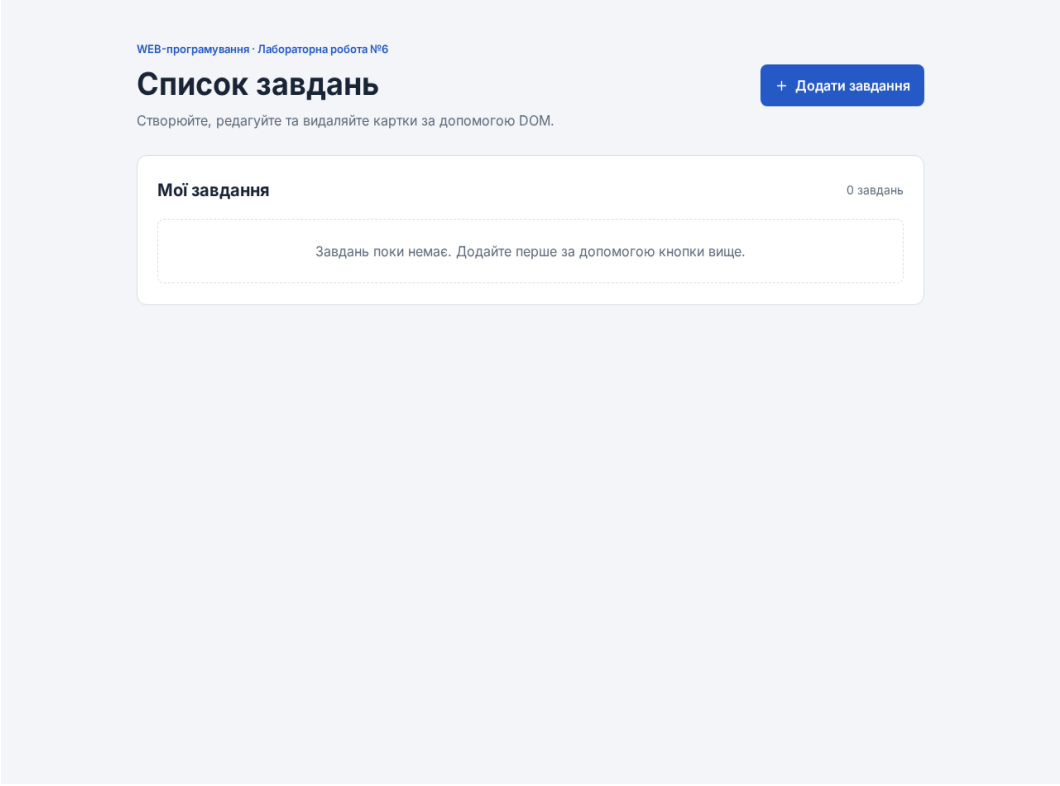



*Рисунок 1 — Початковий інтерфейс із порожнім списком.*

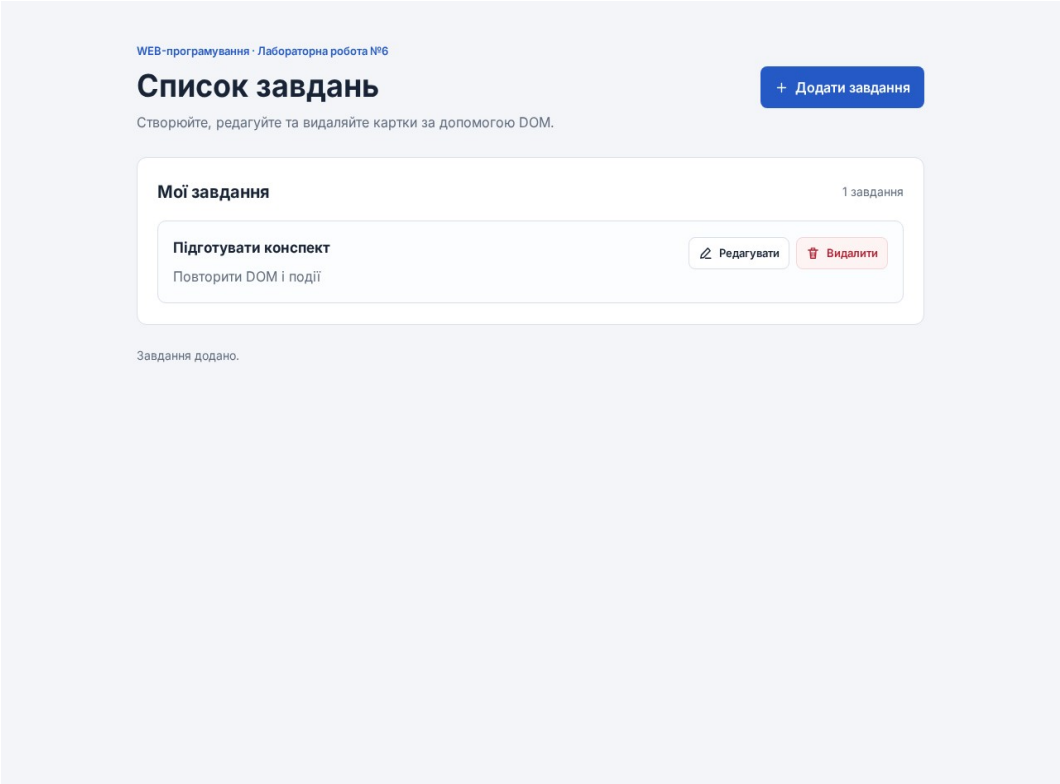



*Рисунок 2 — Картка після створення завдання.*

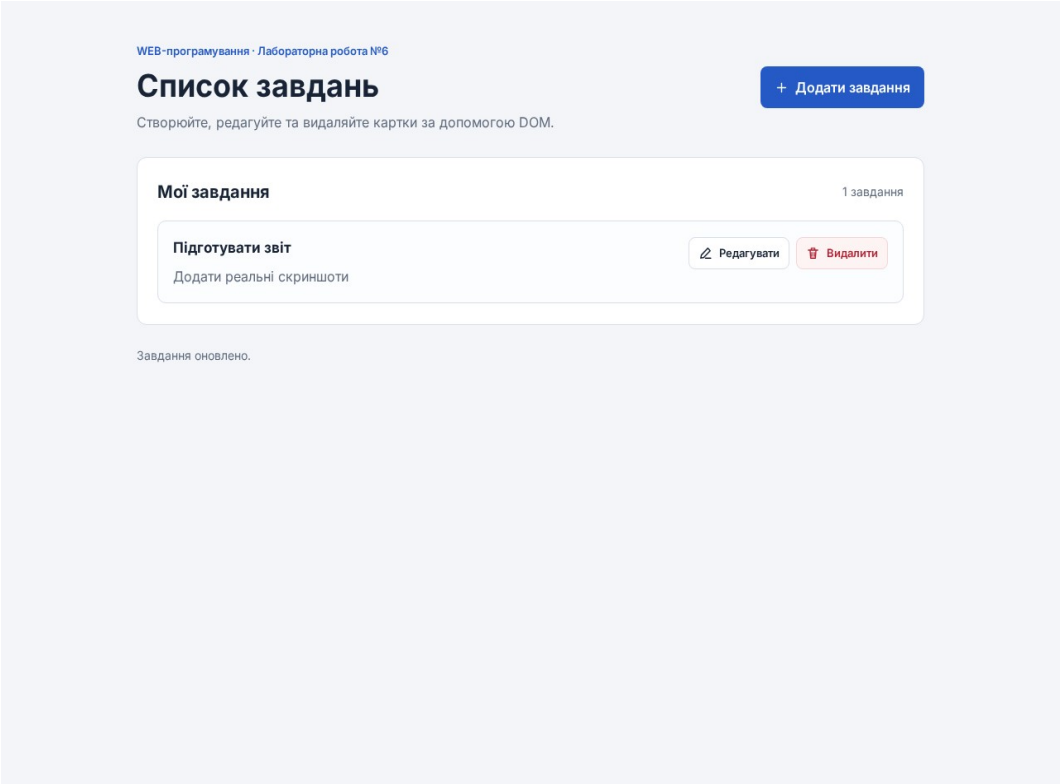



*Рисунок 3 — Картка після збереження нової назви й опису.*

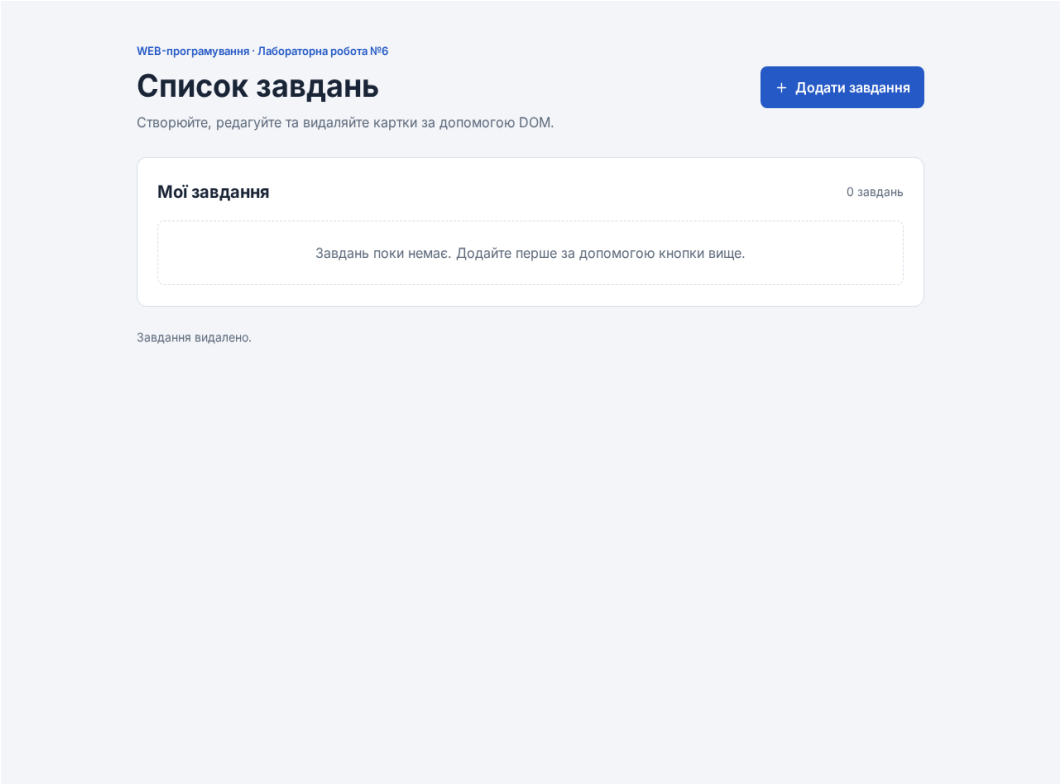



*Рисунок 4 — Картку вилучено з DOM, відновлено порожній стан.*

# 7. Відповіді на питання вихідного контролю

1. **Які події використано?** Document: `DOMContentLoaded`; Form: `submit`; Mouse: `click`.
2. **Для чого `DOMContentLoaded`?** Обробник виконується після побудови DOM-дерева, коли елементи сторінки вже доступні JavaScript.
3. **`target` і `currentTarget`?** `target` — початковий елемент події; `currentTarget` — елемент, обробник якого зараз виконується. При делегуванні це відповідно кнопка або її внутрішня іконка та контейнер списку.
4. **Перевага `template.content.cloneNode(true)`?** Клонується перевірена структура DOM без побудови HTML-рядків; дані можна вставити безпечно через `textContent`.
5. **Навіщо `preventDefault()`?** Він скасовує стандартне надсилання форми й перезавантаження сторінки, щоб дані обробив JavaScript.
6. **Як працює `FormData`?** `FormData(form)` збирає пари ключ-значення з успішних полів. Без атрибута `name` значення поля не потрапить до набору.
7. **Як BEM допомагає JavaScript?** Передбачувані класи `.task-card__title` і `.task-card__desc` дають точні локальні селектори.
8. **Що таке SVG-спрайт?** Це набір повторно використовуваних SVG-символів. `use href="img/sprite.svg#edit"` показує потрібний символ без дублювання геометрії SVG у кожній кнопці.
9. **Чому `submit`, а не `click`?** `submit` охоплює кнопку та клавішу Enter, зберігає семантику форми і використовує її вбудовану валідацію.

# 8. Висновок

Реалізована сторінка відповідає програмній частині методички: створює картки з template, редагує вибрану картку через textContent, видаляє її після підтвердження та використовує делегування подій. Реальні браузерні тести підтвердили всі основні й граничні сценарії. Merge, push та GitHub Pages не виконувалися, оскільки публікація прямо заборонена умовами підготовки архіву.